



One Platform, Many Clients

You do **not** get a separate copy of the app for each client. Instead, one running application serves every client — what changes per client is only their **data and settings**, never the code itself.

✗ NOT THIS (SEPARATE FOLDERS/APPS)



client-a-app/

Own copy of the code, own database



client-b-app/

A separate, disconnected copy



client-c-app/

Yet another copy to maintain

✓ THIS IS HOW IT ACTUALLY WORKS



ONE shared database

One app, one deployment

● Client A — a row of settings

● Client B — a row of settings

● Client C — a row of settings

Why this matters

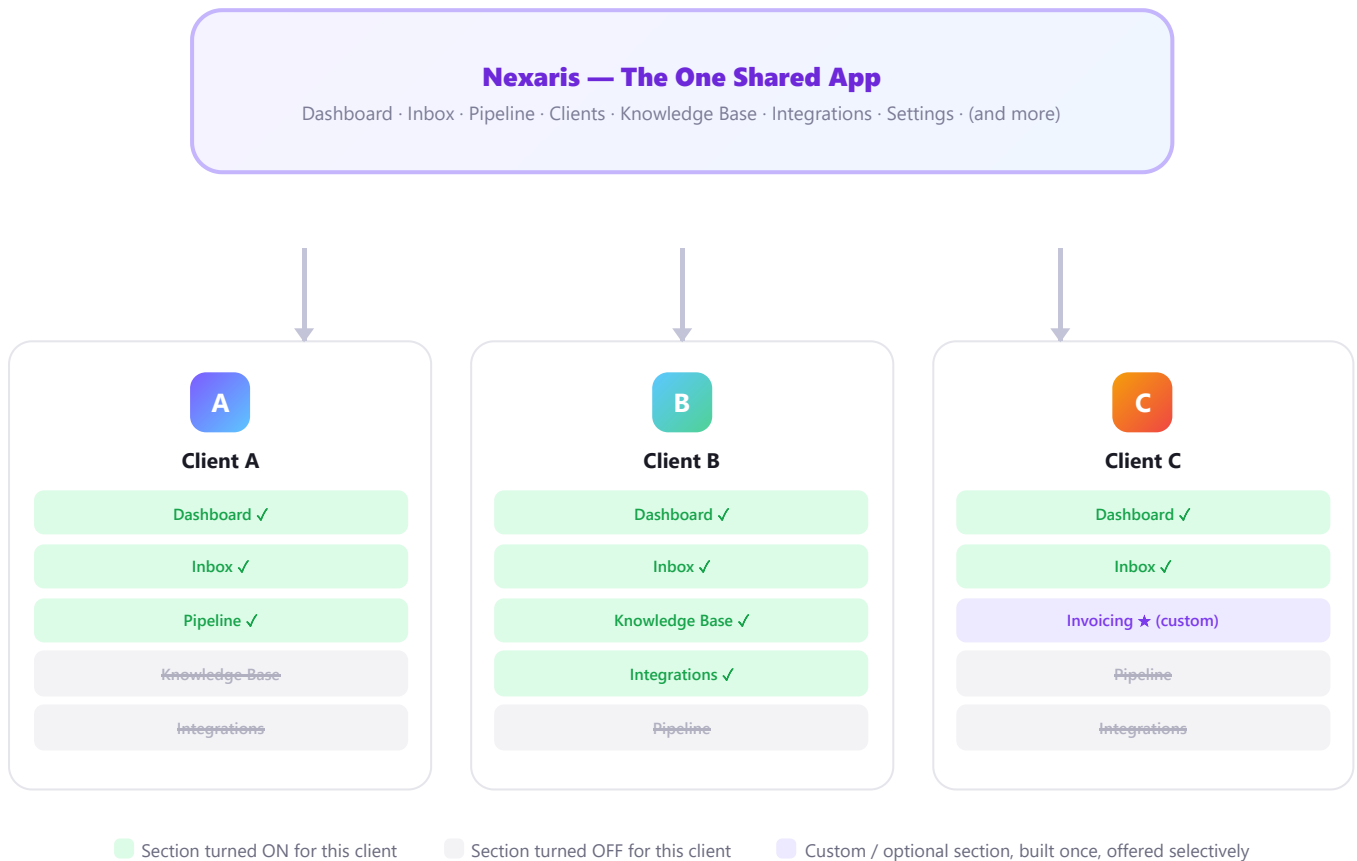
With **separate folders/apps**, every bug fix or improvement has to be repeated in every copy — five clients means five times the maintenance work, and they slowly drift apart until they're basically five different products.

With **one shared app**, you fix a bug or ship a new feature **once**, and every client benefits immediately. What's different per client is stored as **data** (a database row), not as **code** (a separate folder).



Same App, Different Sections Per Client

Every client runs on the exact same app. What's different is which **sections** (menu items / pages) each one sees — controlled by a setting per client, not by different code.



Notice: Client C has an "Invoicing" section that Client A and B don't have. This isn't a separate app built just for Client C — it's a section that exists in the shared platform's library, built once, and simply **switched on** only for the clients who need it. Dashboard and Inbox are "core" sections every client always has.



Adding a New Section for One Client

Here's exactly what happens when a client asks for something that doesn't exist yet in the platform — like Client C's "Invoicing" section from the previous page.

1

Client C requests an Invoicing section

They need to send invoices to their own customers directly from the platform — this doesn't exist in Nexaris today.

2

You (or a developer) build it once, as a normal section

Same as how Pipeline or Knowledge Base were built — a real page inside the shared app, following the same design system and database patterns already in place.

3

It ships turned OFF for everyone by default

The new section exists in the app, but no client sees it in their menu unless it's specifically switched on for them.

☐ Client A — Invoicing: off

☐ Client B — Invoicing: off

4

You switch it ON only for Client C

One click in the admin dashboard — no code change, no redeploy, no separate app.

☒ Client C — Invoicing: **on**

5

Later, Client D wants the same thing

You don't rebuild it — Invoicing already exists in the platform's library. You just switch their toggle on too. The feature pays for itself the more clients use it.

The key idea: every new "custom" section becomes a permanent, reusable part of your platform — not a one-off throwaway build for a single client. Over time you build a **library of optional sections**, and each client simply picks which ones they want switched on, like choosing items from a menu.